AI Lab Assignment 3: Map Coloring using CSP

Constraint Satisfaction Problem • Backtracking Algorithm • Adjacency Representation

Goal & Problem Statement

Assign a color to every state/region so that no two neighbouring states have the same color. This is a classic **Constraint Satisfaction Problem (CSP)**.

1. Is the Code Correct?

Yes, the core logic is correct for a basic lab implementation, provided the neighbour graph is entered consistently as an undirected map (if A neighbours B, then B lists A). It uses backtracking to find a valid assignment from four colors.

Limitations: Trusts input without verifying misspelled/missing names; assumes symmetric relations; does not know real-world borders.

2. Key CSP Concepts

- **Variables:** Each state (e.g., Maharashtra, Goa, Karnataka).
- **Domain:** Available colors: Red, Green, Blue, Brown.
- **Constraints:** Neighbouring states must differ in color.
- **Assignment:** Current color mapping (e.g., `{'A':'Red'}`).
- **Backtracking:** Undo failed choices and try alternative colors.

3. Code Explained in Simple Language

Read Number of States

```
n = int(input("enter no of states: "))
```

Reads how many states/regions the user wants to add.

Set Colors and Assignments

```
colors = ["Red", "Green", "Blue", "Brown"] assigned = {}
```

`colors` forms the domain; `assigned` tracks chosen colors.

Build the Graph

```
graph = {} for i in range(n): state = input("enter the state: ") graph[state] = [] k = int(input("how many neighbours..."))
```

Creates an adjacency list mapping each state to its neighbours.

Check a Color (Constraint Validation)

```
def valid(state, color): for neighbour in graph[state]: if neighbour in assigned and assigned[neighbour] == color: return False return True
```

Validates that no colored neighbour shares the proposed color.

Recursive Solver & Output Execution

```
def solve(index): states = list(graph.keys()) if index == len(states): return True
state = states[index] for color in colors: if valid(state, color): assigned[state] = color if
solve(index + 1): return True del assigned[state] # Backtracking step return False
```

4. Example & Dry Run

Input: 3 mutually connected states (A, B, C)

```
map coloring: A -> Red B -> Green C ->
Blue
```

Dry Run: A takes Red. B checks neighbours, avoids Red, takes Green. C avoids Red and Green, takes Blue.

5. Important Functions

- `valid(state, color)`: Enforces neighbour constraints.
- `solve(index)`: Performs DFS with backtracking.
- `assigned[state] = color`: Makes a tentative choice.
- `del assigned[state]`: Undoes choice on dead ends.

6. Viva Script & 7. Likely Viva Questions

> **Viva Script (30–45s):** "This program solves map coloring using a Constraint Satisfaction Problem. Each state is a variable, and four colors form its domain. The constraint is that neighbouring states must not have the same color. The `valid` function checks conflicts, and `solve` uses backtracking to explore assignments and backtrack upon dead ends."

Question	Answer
What is CSP?	Variables, domains of possible values, and constraints to satisfy.
Why use backtracking?	To undo color choices that lead to dead ends and explore alternatives.
What is the base case in `solve()`?	When `index == len(states)`, meaning all states are successfully assigned.
What is a limitation of this version?	Relies on correct manual input and symmetric adjacency lists without auto-validation.

8. Quick Revision

Summary: Variables = States | Domain = Colors | Constraint = Neighbours Differ | Strategy = Backtracking via `del assigned[state]`.